

Tutorial for Database Setup

0) Environment

CentOS Stream 9

PostgreSQL 17.7 + openGauss 5.0.1

Docker

1) Install Docker Engine (CentOS Stream 9)

```
1  # Remove old packages
2  sudo dnf remove -y docker docker-client docker-client-latest docker-
   common docker-latest docker-latest-logrotate docker-logrotate docker-
   engine || true
3
4  # Install repo tools
5  sudo dnf -y install dnf-plugins-core
6
7  # Add Docker repo
8  sudo dnf config-manager --add-repo
   https://download.docker.com/linux/centos/docker-ce.repo
9
10 # Install Docker + Compose
11 sudo dnf -y install docker-ce docker-ce-cli containerd.io docker-
   buildx-plugin docker-compose-plugin
12
```

```
13 | # Enable and start
14 | sudo systemctl enable --now docker
15 |
16 | # Smoke test
17 | sudo docker run --rm hello-world
```

Allow running Docker without sudo (optional, recommended)

```
1 | # Create group if missing
2 | sudo groupadd docker 2>/dev/null || true
3 |
4 | # Add user to group
5 | sudo usermod -aG docker $USER
6 |
7 | # Refresh group in shell
8 | newgrp docker
9 |
10 | # Verify without sudo
11 | docker run --rm hello-world
```

```
[root@RainYun-I4MYFnsu Project-II]# sudo groupadd docker 2>/dev/null || true
[root@RainYun-I4MYFnsu Project-II]# sudo usermod -aG docker $USER
[root@RainYun-I4MYFnsu Project-II]# newgrp docker
[root@RainYun-I4MYFnsu Project-II]# docker run --rm hello-world
```

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:

<https://hub.docker.com/>

For more examples and ideas, visit:

<https://docs.docker.com/get-started/>

2) Start PostgreSQL + openGauss with Docker Compose

2.1 Create directories and config files

```
1  # Create dirs
2  sudo mkdir -p /opt/dbstack
3  sudo mkdir -p /srv/db/postgres /srv/db/opengauss
4
5  # Set ownership
6  sudo chown -R $USER:$USER /opt/dbstack /srv/db
```

Create `/opt/dbstack/.env` (password is `DBPwd_temp000`):

```
1 cat > /opt/dbstack/.env <<'EOF'
2 # PostgreSQL
3 PG_USER=postgres
4 PG_PASSWORD=DBPwd@temp000
5 PG_DB=postgres
6 PG_PORT=5432
7
8 # openGauss
9 OG_USER=gaussdb
10 OG_PASSWORD=DBPwd@temp000
11 OG_DB=postgres
12 OG_PORT=15432
13 EOF
```

Create `/opt/dbstack/docker-compose.yml` (structure unchanged):

```
1 cat > /opt/dbstack/docker-compose.yml <<'EOF'
2 services:
3   postgres:
4     image: postgres:17
5     container_name: pg
6     environment:
7       POSTGRES_USER: ${PG_USER}
8       POSTGRES_PASSWORD: ${PG_PASSWORD}
9       POSTGRES_DB: ${PG_DB}
10    ports:
11      - "127.0.0.1:${PG_PORT}:5432"
12    volumes:
13      - /srv/db/postgres:/var/lib/postgresql/data:Z
14    restart: unless-stopped
15
16   opengauss:
17     image: enmotech/opengauss:5.0.1
18     container_name: og
19     privileged: true
20     environment:
21       GS_PASSWORD: ${OG_PASSWORD}
22       GS_NODENAME: gaussdb
23       GS_USERNAME: ${OG_USER}
```

```

24     ports:
25         - "127.0.0.1:${OG_PORT}:5432"
26     volumes:
27         - /srv/db/opengauss:/var/lib/opengauss/data:Z
28     restart: unless-stopped
29 EOF

```

Notes:

- PostgreSQL applies `POSTGRES_PASSWORD` only on first init of an empty data dir
- openGauss uses `GS_PASSWORD`
- `:Z` is for SELinux volume labeling

2.2 Start services

```

1  # Start stack
2  cd /opt/dbstack
3  docker compose --env-file .env up -d
4
5  # Show containers
6  docker ps

```

```

[root@RainYun-I4MYFnsu Project-II]# cd /opt/dbstack
[root@RainYun-I4MYFnsu dbstack]# docker compose --env-file .env up -d
[*] up 26/26
✓ Image enmotech/opengauss:5.0.1 Pulled 285.5s
✓ Image postgres:17 Pulled 269.1s
✓ Network dbstack_default Created 9.6s
✓ Container pg Created 8.2s
✓ Container og Created 8.2s
[root@RainYun-I4MYFnsu dbstack]# docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED        STATUS        PORTS                    NAMES
c2dce54922bc   postgres:17        "docker-entrypoint.s..." 7 minutes ago  Up 7 minutes  127.0.0.1:5432->5432/tcp  pg
9767c254ff0f   enmotech/opengauss:5.0.1 "entrypoint.sh gauss..." 7 minutes ago  Restarting (1) About a minute ago

```

3) One-liners: Login and run any local `.sql`

3.1 PostgreSQL: one-line login (`psql`)

```

1  docker exec -it pg psql -U postgres -d postgres

```

3.2 PostgreSQL: one-line run a local `.sql`

```
1 | docker exec -i pg psql -U postgres -d postgres -v ON_ERROR_STOP=1 -1 < /any/path/your.sql
```

3.3 openGauss: one-line login (`gsql`)

```
1 | docker exec -it og bash -lc "su - omm -c 'gsql -d postgres -p 5432'"
```

3.4 openGauss: one-line run a local `.sql`

```
1 | docker exec -i og bash -lc "su - omm -c 'gsql -d postgres -p 5432 -v ON_ERROR_STOP=on -1 -f -' < /any/path/your.sql"
```

4) Host shortcuts: `pglogin` / `pgsql` / `oglogin` / `ogsq1`

4.1 Install scripts into `/usr/local/bin`

```
1 | sudo tee /usr/local/bin/pglogin >/dev/null <<'EOF'
2 | #!/usr/bin/env bash
3 | set -euo pipefail
4 | DB="${1:-postgres}"
5 | USER="${PGUSER:-postgres}"
6 | docker exec -it pg psql -U "$USER" -d "$DB"
7 | EOF
8 | sudo chmod +x /usr/local/bin/pglogin
```

```

1  sudo tee /usr/local/bin/pgsql >/dev/null <<'EOF'
2  #!/usr/bin/env bash
3  set -euo pipefail
4  FILE="${1:?Usage: pgsql /path/to/file.sql [dbname]}"
5  DB="${2:-postgres}"
6  USER="${PGUSER:-postgres}"
7  docker exec -i pg psql -U "$USER" -d "$DB" -v ON_ERROR_STOP=1 -1 <
   "$FILE"
8  EOF
9  sudo chmod +x /usr/local/bin/pgsql

```

```

1  sudo tee /usr/local/bin/oglogin >/dev/null <<'EOF'
2  #!/usr/bin/env bash
3  set -euo pipefail
4  DB="${1:-postgres}"
5  docker exec -it og bash -lc "su - omm -c 'gsq1 -d $DB -p 5432'"
6  EOF
7  sudo chmod +x /usr/local/bin/oglogin

```

```

1  sudo tee /usr/local/bin/ogsq1 >/dev/null <<'EOF'
2  #!/usr/bin/env bash
3  set -euo pipefail
4  FILE="${1:?Usage: ogsq1 /path/to/file.sql [dbname]}"
5  DB="${2:-postgres}"
6  docker exec -i og bash -lc "su - omm -c 'gsq1 -d $DB -p 5432 -v
   ON_ERROR_STOP=on -1 -f -' < \"$FILE\"
7  EOF
8  sudo chmod +x /usr/local/bin/ogsq1

```

4.2 Usage examples (from any directory)

```

1  pglogin
2  pgsql ./migrate.sql mydb
3
4  oglogin
5  ogsq1 /home/me/sql/init.sql postgres

```

5) Quick self-check

```
1 | # PostgreSQL version
2 | docker exec -i pg psql -U postgres -d postgres -c "select version();"
```

```
1 | # openGauss version
2 | docker exec -i og bash -lc "su - omm -c \"gsql -d postgres -p 5432 -c
   | 'select version();'\""
```

6) Common issues and fixes

1. `.env` changed but password did not change

Images usually apply env vars only on first init of an empty data dir.

- `docker compose down`
- remove data dir: `/srv/db/postgres` or `/srv/db/opengauss`
- `docker compose up -d`

2. Port conflicts

- PostgreSQL: `127.0.0.1:5432`
- openGauss: `127.0.0.1:15432`

3. Local-only access (safer default)

Ports are bound to `127.0.0.1`. Remove `127.0.0.1:` if you need LAN access.